Eg: Array insert_last() growing array

pass 1: [1] increase twice so size = 2

[1|2| |] size = 4

& so on size = 1, 2, 4, 8, 16 (2^0 for nothing)

→ Worst cost = $\Theta(1+2+4+8+16+\dots+n)$

$= \Theta\left(\sum_{i=1}^{\log n} 2^i\right) = \Theta(2^{\log n}) = \Theta(n) \approx$ Amortized $O(1)$

in geometric sequence, where seq is growing exponentially, we can simply pick the last item

Amortization

Operation takes $T(n)$ amortized time if any k operations take $\leq kT(n)$ time

or simply → "averaging over operation sequence" ← Total time it takes to do n operations cannot take more than n time — over the long term, this is on the average

Sequence Interface

Data Structures	Static: get_at(i) set_at(i,x)	Dynamic: insert_first(x) delete_first()	Dynamic: insert_last(x) delete_last()	Dynamic: insert_at(i,x) delete_at(i)
Array	1	n	n	n
Linked list	n	1	n	n
dynamic array	1	n	1 (a)	n

Lecture 3 (Introduction to Algorithms – Problem session 1 – Asymptotic Behavior of Functions & Double ended)

$\sqrt{n} < n < n+\sqrt{n}$

$\log(n) = o(n^b)$

Binomial coefficient

$\binom{n}{k} = \frac{n!}{k!(n-k)!}$

no of n ways to do k

How big is $n!$

→ Stirling's approximation

$\sqrt{2\pi n}\left(\frac{n}{e}\right)^n$

so, $\log(n!) = \log\left(\sqrt{2\pi n}\left(\frac{n}{e}\right)^n\right) \to \Theta(n \log n)$

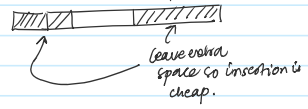
Implementation of an interface is array, linked list etc

Q. Swap_ends (Data structure) using data structure specific operations

```
swap_ends(arr):
    x1 = arr.delete_first()
    x2 = arr.delete_last()

    arr.insert_first(x2)
    arr.insert_last(x1)
```

You could use temp and simply swap



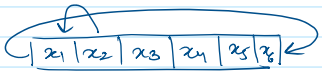
Dynamic array has doubly ended space.

What about extra space? – At no point we should have extra space & waste it

" Keep some space always "

Q. shift left

shift_left(...) in O(k)



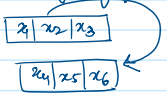
put x_1 behind x_6 and x_2 to x_1

Brute force

delete x_1 & insert at x_6 for n times?

x_2 is naturally at x_1 now?

Let's say shift from x_3



So let's split the array & reverse the first half and shift it at the end?

Recursive!

```
if k==0 or k==1:
    return
x = delete_first()
insert_last(x)
shift_left(0, k-1)
```

↖ array indices